System Design Specification: Modular Base CSE and Multi-Instance Vehicle AE Architecture

Doc ID: SDS-SDV-MODULAR-001 **Status:** Approved Technical Architecture **Standard:** oneM2M TS-0001, TS-0004, TS-0010

Security: RFC 8446 (TLS 1.3 mTLS)

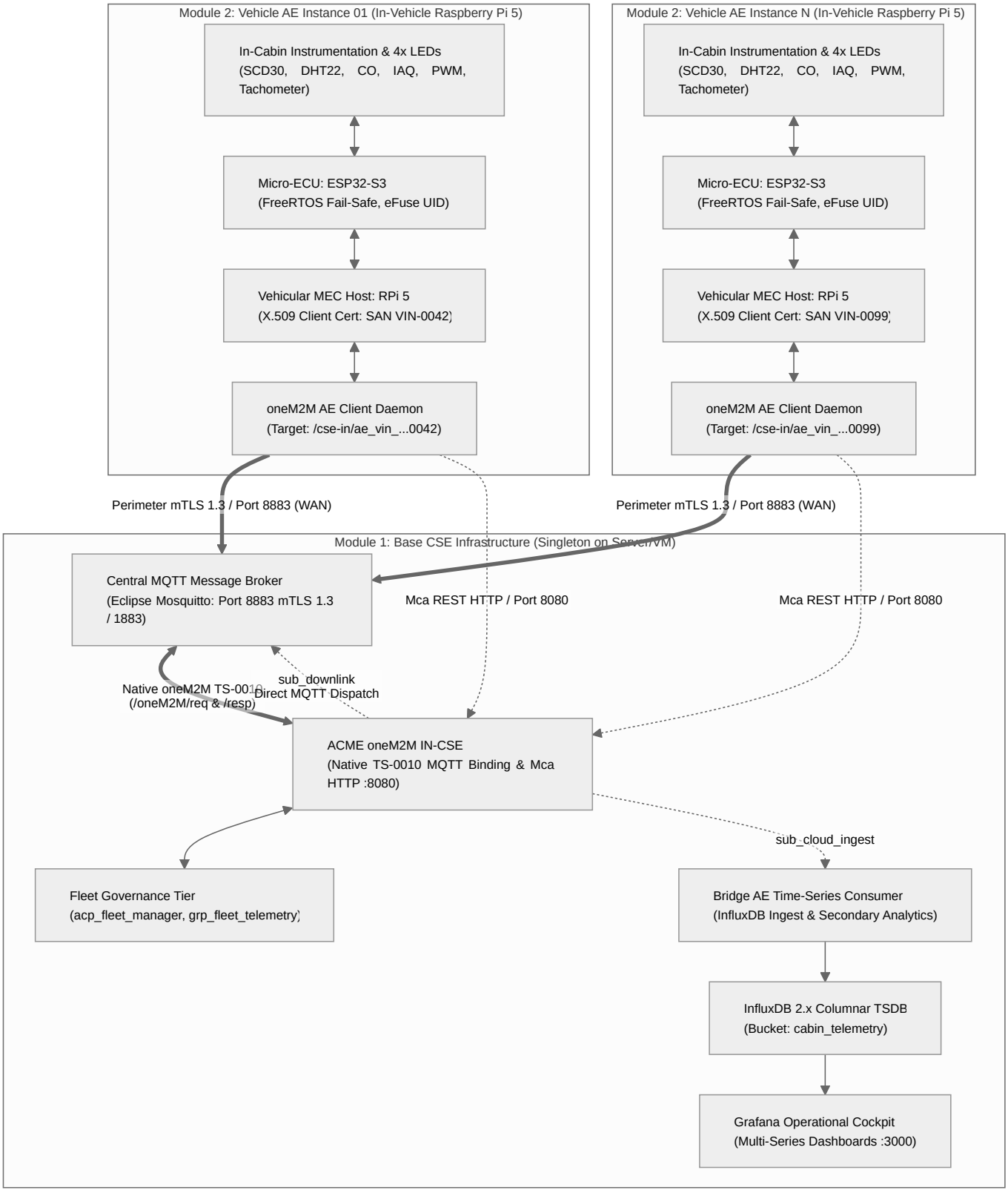
Document ID: SDS-SDV-MODULAR-001
Version: 2.1.0 (Physical Host Allocation & Consolidated Vehicular Edge Gateway Architecture)
Status: Approved Technical Architecture
Project: Software-Defined Vehicle (SDV) Automotive IoT Cabin Monitoring & HVAC Regulation
Standard Compliance: ETSI TS 118 101 Rel 4, oneM2M TS-0001, oneM2M TS-0003, oneM2M TS-0004, oneM2M TS-0010 (MQTT Protocol Binding)
Associated Baseline: acme_cse_gateway_capability_review.md

1. EXECUTIVE SUMMARY AND ARCHITECTURAL PRINCIPLES

This specification defines the structural partitioning, physical host allocations, interface contracts, identity-binding protocols, and deployment lifecycle for the SDV Cyber-Physical Environmental Monitoring and Closed-Loop HVAC Regulation platform.

The architecture enforces a strict demarcation between the central infrastructure platform and distributed vehicular edge compute nodes:

1. **Base CSE Infrastructure Module (Singleton Instance — Server / Virtual Machine):** * **Host Platform:** Deployed on an external connected device—a **central physical server, cloud instance, or virtual machine (VM)**. * **Role:** Serves as the authoritative Common Services Entity (ACME oneM2M IN-CSE), central MQTT message broker (Eclipse Mosquitto terminating perimeter mTLS 1.3), fleet governance policy repository, time-series data lake (InfluxDB 2.x), and fleet-wide operational cockpit (Grafana Enterprise). * **Cardinality: Strictly a single, authoritative instance per operational cluster ($C = 1$).**
2. **Vehicle Application Entity (AE) Module (Multi-Instance — In-Vehicle Raspberry Pi 5):** * **Host Platform:** Deployed **directly onboard each physical vehicle on a dedicated Raspberry Pi 5** single-board computer (vehicular MEC host). * **Role:** Interfaced directly with in-cabin physical instrumentation (ESP32-S3 micro-ECU, Sensirion SCD30, DHT22, CO, IAQ/VOC, AQI transducers, 4-channel LED actuator matrix, WS2812B annunciator, and closed-loop tachometer). The Raspberry Pi 5 executes the uniquely identified Application Entity (AE) client daemon, local sensor ingress/pre-processing, mTLS client credentials, and downlink actuation dispatcher. * **Cardinality: Deployable across N uniquely identified, concurrent, and independent vehicular instances ($i \in \{1, 2, \dots, N\}$).**



2. CONSOLIDATION OF VEHICULAR EDGE GATEWAY FUNCTIONS INTO ACME ONEM2M CSE

As established in the architectural capability review (acme_cse_gateway_capability_review.md), the platform consolidates legacy edge gateway microservices directly into native ETSI TS 118 101 middleware capabilities.

2.1 Excised Microservices and Absorbed Capabilities

1. **Decommissioning of Node-RED Ingress Proxy (`nodered_ipe`):** * *Legacy Role:* Acted as an intermediate Node.js container translating raw MQTT topics into HTTP REST `<m2m:cin>` instances, and translating HTTP webhooks into downlink MQTT actuation commands. * *Consolidated Architecture:* Completely decommissioned and excised from the deployment mesh. ACME oneM2M IN-CSE natively terminates and dispatches oneM2M primitives over MQTT via **oneM2M TS-0010 MQTT protocol bindings** (`[mqtt]` in `deploy/onem2m/acme.ini`). The vehicle node publishes ContentInstances directly to `/oneM2M/req/...` and receives actuation

downlinks on `/oneM2M/req/id-in/{aei}/json`. * *Performance Gain:* Reclaims ~220 MB RAM and reduces closed-loop actuation propagation latency by $40\text{--}80\ \text{ms}$.

2. **In-CSE Multi-Hazard Health Classification:** * *Legacy Role:* The standalone Python `bridge_ae` microservice evaluated all sensor divergence, stale timestamps, and motor stalls. * *Consolidated Architecture:* ACME CSE's internal scripting engine (`acme-scripts` with `onCreate` hooks) absorbs multi-hazard rule evaluation directly upon ContentInstance creation. The CSE tags `<cin>` resources with semantic labels (`lbl: ["health:FRESH", "delta_t:0.32", "delta_rh:1.8", "stall:false"]`) and raises emergency alarms internally. `bridge_ae` is streamlined into a lightweight, non-blocking time-series persistence consumer.
3. **Edge Time-Series Buffering and Storage Caps:** * *Consolidated Architecture:* Rather than running a heavyweight edge TSDB for short-term buffering, ACME CSE enforces standard oneM2M container lifecycle bounds:
   - `mni: 1440`: Buffers 1,440 historical instances (24 hours at 1-minute cadence).
   - `mbs: 2097152`: Strict 2 MB memory-footprint cap protecting vehicular storage.
   - `mia: 86400`: 24-hour automatic TTL instance pruning.
   - `missingDataDetect: true`: Built-in sensor loss detection without custom watchdog daemons.
4. **Dynamic Fleet Group Fan-Out (`<grp>`, `<fopt>`):** * *Consolidated Architecture:* Centralised group aggregation allows atomic fleet status queries (`/grp_fleet_telemetry/fopt/la`) and multi-vehicle broadcast actuations without external orchestrator scripts.
5. **Dynamic Semantic Policy Engine (`<acp>`):** * *Consolidated Architecture:* Replaces static broker ACL text files with standard oneM2M Access Control Policies, dynamically binding privileges to verified AE-IDs.

2.2 Architectural Segregation: Retained Dedicated Platforms

To guarantee cyber-physical safety, cryptographic robustness, and high-performance analytics, specific functions are strictly retained on dedicated platforms outside the CSE:

Subsystem	Placement	Architectural Rationale
Mutual TLS 1.3 Termination	**Eclipse Mosquitto (Port 8883)**	Cryptographic perimeter security, hardware ECDSA P-256 acceleration, and X.509 client certificate verification must terminate at a hardened C broker, protecting the Python ASGI/WSGI stack.
Long-Term Multi-Series TSDB	**InfluxDB 2.x Columnar Engine**	High-cardinality nanosecond time-series analytics and Flux queries across months of fleet operation exceed the capacity of oneM2M SQLite/TinyDB metadata backends.
Operational Cockpit	**Grafana Enterprise (Port 3000)**	Fleet-wide visual observability, multi-series dashboards, dynamic alerting, and operator control cockpits.
Autonomous Safety Overrides	**ESP32-S3 FreeRTOS Firmware**	Safety-critical fail-safe loops (e.g. 50% baseline purge ventilation during communication loss or CO emergency) execute deterministically in microcontroller firmware independent of edge host or network state.

3. MODULE 1: BASE CSE INFRASTRUCTURE SPECIFICATION (SINGLETON ON SERVER / VIRTUAL MACHINE)

3.1 Host Platform and Deployment Topology

- **Host Platform:** Dedicated physical server, cloud instance, or virtual machine (VM) (Linux x86_64 or aarch64 server environment).
- **Singleton Constraint:** Exactly one Base CSE infrastructure instance exists per operational cluster. It operates with a stable, routable network identity (e.g. `GATEWAY_IP=192.168.122.91` or cluster FQDN).
- **Deployment Packaging:** Packaged and deployed via `deploy/docker-compose.yml` on the central host.

3.2 Core Component Responsibilities

1. **ACME oneM2M IN-CSE (`sdv_onem2m_cse`):** * Exposes standard Mca HTTP(S) reference point on port `8080` and native oneM2M TS-0010 MQTT binding on internal port `1883`. * Manages the root `<m2m:cb>` (CSEBase) identified by `cse_id` (`id-in`) and accessible at path `/{{ cse_rn }}` (`/cse-in` or `/sdv-cse`). * Enforces semantic schema validation, resource lifecycles, and access control policies.
2. **Fleet Governance Layer (Pre-Provisioned Hierarchy):** * **Fleet Access Control Policy (`acp_fleet_manager`):** Establishes default administrative and onboarding privileges. * **Fleet Fan-Out Group (`grp_fleet_telemetry`):** An aggregate `<m2m:grp>` whose member IDs (`mid`) dynamically index the raw telemetry containers of all onboarded vehicles. Supports atomic fleet queries via `/grp_fleet_telemetry/fopt/la`. * **Downlink & Cloud Ingestion Subscriptions:** Dispatches asynchronous change events to subscriber targets.
3. **Data Health & Storage Pipeline:** * **Central Mosquitto Broker (`sdv_mosquitto_broker`):** Terminates network mTLS 1.3 on port 8883 from all vehicular Raspberry Pi 5 nodes. * **Bridge AE Time-Series Consumer (`sdv_bridge_ae`):** Consumes `<subscription>` notifications and writes clean records to InfluxDB. * **InfluxDB 2.x TSDB (`sdv_influxdb`):** Historical time-series persistence. * **Grafana Operational Cockpit (`sdv_grafana`):** Observability and fleet-wide KPI visualisation.

3.3 Initial State Guarantee

Upon initial bootstrap, the Base CSE module **must contain zero vehicular Application Entities**. No vehicle node shall be present until dynamically onboarded by an authentic Vehicle AE module running on an in-vehicle Raspberry Pi 5.

---

4. MODULE 2: VEHICLE AE SPECIFICATION (MULTI-INSTANCE ON IN-VEHICLE RASPBERRY PI 5)

4.1 Host Platform and Deployment Topology

- **Host Platform:** Dedicated **Raspberry Pi 5 single-board computer** installed inside each individual vehicle.
- **Multi-Instance Constraint:** For N vehicles in the fleet, N distinct Raspberry Pi 5 units are deployed. Each unit operates completely independently with zero shared local state, executing its own isolated edge runtime.
- **Network Connection to Base CSE:** Connects over vehicular wide-area networks (cellular 4G/5G, road-side Wi-Fi, or vehicular mesh) back to the central Server/VM on port 8883 (mTLS 1.3) and port 8080 (oneM2M Mca).

4.2 Physical and Edge Compute Stack

Each in-vehicle Raspberry Pi 5 orchestrates two physical and computational tiers:

Tier A: Physical Sensing & Actuation Node (ESP32-S3 Micro-ECU)

- **Hardware Interface to Raspberry Pi 5:** Directly connected via a dedicated physical serial bus (USB-CDC on `/dev/serial/by-id/usb-Espressif_ESP32-S3_...` or hardware GPIO UART) with parity checking.
- **Transducers (7 Environmental Channels):**
- Sensirion SCD30: NDIR CO_2 (0–10,000 ppm), dry-bulb temperature, relative humidity over $\mathrm{I}^2\mathrm{C}$ Fast-Mode (400 kHz) with hardware CRC-8 verification.
- DHT22: Redundant temperature and relative humidity over 1-Wire bus with parity checksum.
- Carbon Monoxide (CO) Sensor: Toxic gas detection (0–1,000 ppm) with emergency alert interrupt.
- Cabin Air Quality (IAQ/VOC): Volatile organic compound index (0–500) for passenger air quality.
- Exterior Air Quality (AQI): Ambient outdoor index for fresh-air intake vs cabin air recirculation damper control (ΔIAQ).
- **Actuators (4-Channel Emulation Matrix & Closed-Loop Tachometer):**
- Blue LED (GPIO 18): Purge blower ventilation driven at 25.0 kHz PWM with 10-bit resolution (0--100% duty cycle).
- Red LED (GPIO 17): Auxiliary cabin PTC heating core driven by digital output ($T_{\text{cabin}} < 19.0\,^{\circ}\text{C}$).
- Green LED (GPIO 16): Baseline eco-ventilation driven by digital output (nominal air quality).
- Amber LED (GPIO 15): Dehumidification and warning alert driven by digital output ($RH > 65\%$ or degraded data health).
- WS2812B RGB LED (GPIO 38): Optical state annunciator indicating network, sensor, and safety status.
- Hall-Effect Tachometer Feedback (GPIO 19): Interfaced via Pulse Counter (PCNT) Unit 0 for closed-loop blower stall detection.
- **Firmware Runtime:** FreeRTOS with strict core affinity (Core 1: Sensor/PWM loop; Core 0: mTLS/Network loop).
- **Hardware Identity:** Immutable 48-bit MAC address and 128-bit UID read from write-once eFuse registers.

Tier B: Vehicular MEC Host (Raspberry Pi 5)

- **Compute Platform:** Quad-core ARM Cortex-A76 vehicular edge host running vehicular Linux.
- **Local Ingress & Formatting Agent:** Ingests raw telemetry frames from the ESP32-S3, verifies CRC-8/parity checksums, and serialises valid oneM2M JSON content instances.
- **Cryptographic Identity Vault:** Securely stores the vehicle's unique ISO 3779 VIN, OTP board serial, and private key/X.509 client certificate for mTLS 1.3 authentication against the central server/VM.
- **oneM2M AE Client Agent:** Executes the onboarding reconciliation loop, periodic telemetry transmission to the Base CSE via native TS-0010 MQTT or Mca REST, and actuation downlink command dispatch to the ESP32-S3.

4.3 Dynamic oneM2M Resource Representation

When a vehicle's Raspberry Pi 5 registers with the Base CSE on the server/VM, it provisions a dedicated subtree under `<m2m:cb>`:

```
/cse-in (CSEBase on Server/VM)
 └── ae_vin_wauzzz_sdv_2026_0042 (Application Entity: <m2m:ae> deployed on RPi 5)
      ├── temperature-cabin (Container: <m2m:cnt>)
      ├── humidity-cabin (Container: <m2m:cnt>)
      ├── co2-cabin (Container: <m2m:cnt>)
      ├── co-cabin (Container: <m2m:cnt>)
      ├── air-quality-cabin (Container: <m2m:cnt>)
      ├── air-quality-exterior (Container: <m2m:cnt>)
      ├── raw-telemetry (Container: <m2m:cnt>)
      │    ├── cin_xxxx (<m2m:cin>)
      │    └── sub_cloud_ingest (<m2m:sub>)
      └── commands (Container: <m2m:cnt>)
           └── sub_downlink_actuation (<m2m:sub>)
```

5. MULTI-LAYER BINDING MECHANISM: UNIQUELY LINKING VEHICLE TO AE

To guarantee that each physical vehicle solely and accurately links to its designated AE instance on the Base CSE without cross-talk or collision, the 5-layer binding mechanism operates across the module boundary:

Layer	Binding Mechanism	Implementation Across Physical Hosts
1. Physical Hardware	ESP32 eFuse MAC + RPi 5 Board Serial $\rightarrow$ ISO 3779 VIN	**In-Vehicle RPi 5:** Embedded in local device tree and sensor frame headers.
2. Transport Security	X.509 Mutual TLS 1.3 with Subject Alternative Name (SAN)	**Network Boundary:** RPi 5 presents client cert with `SAN: URI:urn:sdv:vin:<VIN>`. Server/VM gateway authenticates cert against vehicular Root CA.
3. Semantic Identity	Deterministic Resource Name ($rn = \text{FNV-1a}(\text{VIN})$)	**Middleware Boundary:** RPi 5 registers/reconciles deterministic rn. Server/VM Base CSE binds allocated aei to client credential.
4. Access Control	Dedicated `<m2m:acp>` per vehicle ($acop = 63$ for aei)	**Server/VM Base CSE:** Restricts CRUD operations on `/ae_vin_<slug>/*` strictly to that vehicle's authenticated aei and $CAdmin$.
5. Downlink Routing	Targeted Point of Access (poa) & MQTT topics	**Network Boundary:** Actuator directives route strictly to `sdv/{vin}/control/cmd` or native topic `/oneM2M/req/id-in/{aei}/json`.

5.1 Deterministic Resource Name Equation

To prevent collisions and eliminate hardcoded aliases, the AE Resource Name is mathematically derived from the vehicle's unique identity string:

$$rn_{\text{AE}} = \text{ae_vin_} \parallel \text{slugify}\ (\text{VIN}) \parallel \text{_} \parallel \text{hex}\ \left(\ \ FNV-1a_{32}(\text{VIN})\ \right)$$

This guarantees: 1. **Uniqueness:** Two distinct vehicles ($VIN_A \neq VIN_B$) yield distinct resource names ($rn_A \neq rn_B$). 2. **Idempotence:** A rebooting vehicle always resolves the exact same resource name, eliminating duplicate "ghost" AEs.

6. MULTI-DIMENSIONAL DATA HEALTH CLASSIFICATION ENGINE

The system evaluates incoming sensor telemetry against five quantitative health criteria:

$$\Delta T = \ | T_{\text{SCD30}} - T_{\text{DHT22}} \ | \ \leq 1.5\,^{\circ}\text{C}$$

$$\Delta RH = \ | RH_{\text{SCD30}} - RH_{\text{DHT22}} \ | \ \leq 10.0\,\%$$

$$\tau_{\text{age}} = t_{\text{eval}} - t_{\text{sample}} \leq 2.5\,\text{s}$$

$$\text{Stall Fault} \iff (\text{PWM} \geq 30\%) \land (\text{RPM} < 100)$$

$$\text{Toxic Gas Hazard} \iff \text{CO} \geq 30.0\,\text{ppm}$$

Categorical State Assignment

1. **`FRESH`**: $\tau_{\text{age}} \leq 2.5\,\text{s}$, $\Delta T \leq 1.5\,^{\circ}\text{C}$, $\Delta RH \leq 10\%$, and tachometer feedback confirmed. Full closed-loop regulation active.
2. **`STALE`**: $2.5\,\text{s} < \tau_{\text{age}} \leq 10.0\,\text{s}$. Network latency delay; hold-last-valid-value regulation logic engaged.
3. **`DEGRADED`**: $\Delta T > 1.5\,^{\circ}\text{C}$ or $\Delta RH > 10.0\%$. Transducer divergence; amber annunciator illuminated; system relies on conservative SCD30 readings.
4. **`FAULT`**: $\tau_{\text{age}} > 10.0\,\text{s}$, null reading, out-of-bounds reading, or blower motor stall detected. Numerical values excluded from time-series aggregates; blower defaults to 50% baseline fail-safe ventilation.

7. DEPLOYMENT AND OPERATIONAL SCENARIOS

7.1 Scenario A: Singleton Base CSE Bootstrap (Server / Virtual Machine)

1. The cloud or server infrastructure deploys `deploy/docker-compose.yml` on the Server/VM.
2. `sdv_onem2m_cse`, `sdv_mosquitto_broker`, `sdv_bridge_ae`, `sdv_influxdb`, and `sdv_grafana` initialize (Node-RED is decommissioned).
3. The Base CSE governance bootstrap script (`deploy_acme_fleet_cse_base.py`) executes: * Verifies CSEBase reachability (`GET /cse-in`). * Provisions `acp_fleet_manager` (). * Provisions `grp_fleet_telemetry` () with an empty member array (`mid = []`). * **Result:** Clean, production-ready Base CSE with zero registered vehicle AEs.

7.2 Scenario B: Commissioning a New Vehicle AE Instance (In-Vehicle Raspberry Pi 5)

Each physical vehicle executes its onboarding sequence independently from its onboard Raspberry Pi 5: 1. **Hardware Power-On:** ESP32 boots FreeRTOS, begins sensor sampling, and establishes local bus communication with the Raspberry Pi 5. 2. **Identity Resolution:** Raspberry Pi 5 agent reads local VIN and computes deterministic rn_{AE}. 3. **Reconcile-on-Boot Check:** * Raspberry Pi 5 issues `GET /cse-in/{{ vehicle_ae_rn }}` over WAN to the Server/VM with `X-M2M-Origin: CAdmin` (or authenticated client cert). * **Branch 1 (New Vehicle - 404 Not Found):** * Issues `POST /cse-in` with `ty=2` to create `<m2m:ae>` using deterministic rn. * Iterates through the canonical sensor manifest to create the 8 standard containers (`<m2m:cnt>`). * Attaches downlink subscription to `/commands`. * Registers its `raw-telemetry` container in Base CSE's `grp_fleet_telemetry.mid`. * **Branch 2 (Reconnecting Vehicle - 200 OK):** * Validates existing container tree. * Reclaims its assigned `aei` and immediately resumes telemetry commits. 4. **Steady-State Ingestion:** Raspberry Pi 5 posts telemetry instances (`<m2m:cin>`) via native oneM2M TS-0010 MQTT or Mca REST directly to its own `/raw-telemetry` container on the Server/VM.

7.3 Scenario C: Decommissioning / Retiring a Vehicle AE Instance

When a vehicle is decommissioned or retired from the fleet: 1. Administrative command or vehicle departure agent issues `DELETE /cse-in/{{ vehicle_ae_rn }}` against the Server/VM. 2. Base CSE cascade-deletes the vehicle's `<m2m:ae>`, all 8 child `<m2m:cnt>` containers, all `<m2m:cin>` records, and subscriptions. 3. Governance reconciliation script removes the deleted container URI from `grp_fleet_telemetry.mid`. 4. Other active vehicle AE instances running on other Raspberry Pi 5 units remain entirely unaffected.

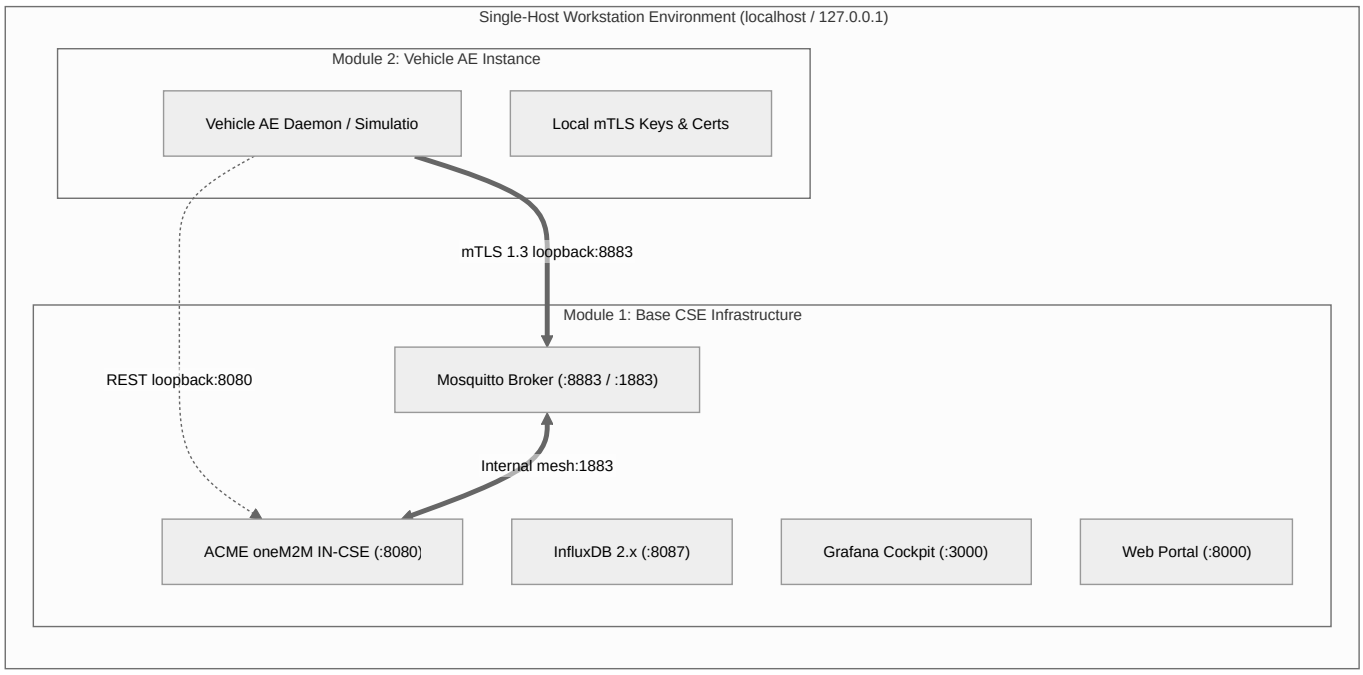
---

8. DEPLOYMENT TOPOLOGY OPTIONS: LOCALHOST AND DISTRIBUTED REMOTE HOSTS

To address diverse operational environments across the vehicle engineering lifecycle—ranging from local workstation software development and continuous integration to in-vehicle road testing and cloud-hosted fleet management—the platform explicitly specifies and supports two operational deployment topologies.

8.1 Topology Option A: Co-Located Localhost Topology (`DEPLOY_TARGET=localhost`)

The **Localhost Deployment Topology** co-locates Module 1 (Base CSE Infrastructure) and Module 2 (Vehicle Application Entity) upon a single physical or virtual developer workstation.



Characteristics and Operational Objectives

1. **Target Environment:** Developer laptops, continuous integration (CI) runners, and offline simulation nodes.
2. **Network Demarcation:** All inter-module communication is strictly contained within the loopback adapter (`127.0.0.1` / `localhost`). Zero wide-area network routing or external IP connectivity is required.
3. **Automated Reachability Fallback:** All deployment and verification scripts (`deploy_gateway.sh`, `init_base_cse.sh`, `validate_base_cse.py`, `deploy_acme_fleet_single_vehicle_ae.py`) automatically detect when `DEPLOY_TARGET=localhost` and bind to local endpoints without waiting for external network timeouts.
4. **Execution Command:** `bash make menu # Interactive terminal dashboard (Option [4]) # Or direct CLI invocation: make deploy-localhost # Equivalent to: bash deploy/deploy_gateway.sh --localhost --lean`

8.2 Topology Option B: Distributed Remote Hosts Topology (`DEPLOY_TARGET=remote`)

The **Distributed Remote Hosts Topology** implements the physical and operational demarcation between the central fleet infrastructure and distributed vehicular edge computing nodes deployed across two distinct remote hosts: * **Remote Host 1 (Base CSE Host):** Dedicated central physical server or virtual machine identified by `BASE_CSE_HOST` (e.g. `192.168.122.91` or `base-cse.sdv.cluster.local`). * **Remote Host 2 (Vehicle AE Host):** In-vehicle Raspberry Pi 5 single-board computer identified by `VEHICLE_AE_HOST` (e.g. `192.168.122.105` or `rpi5-car01.sdv.vehicle.local`).

Detailed Host Characteristics and Separation of Concerns

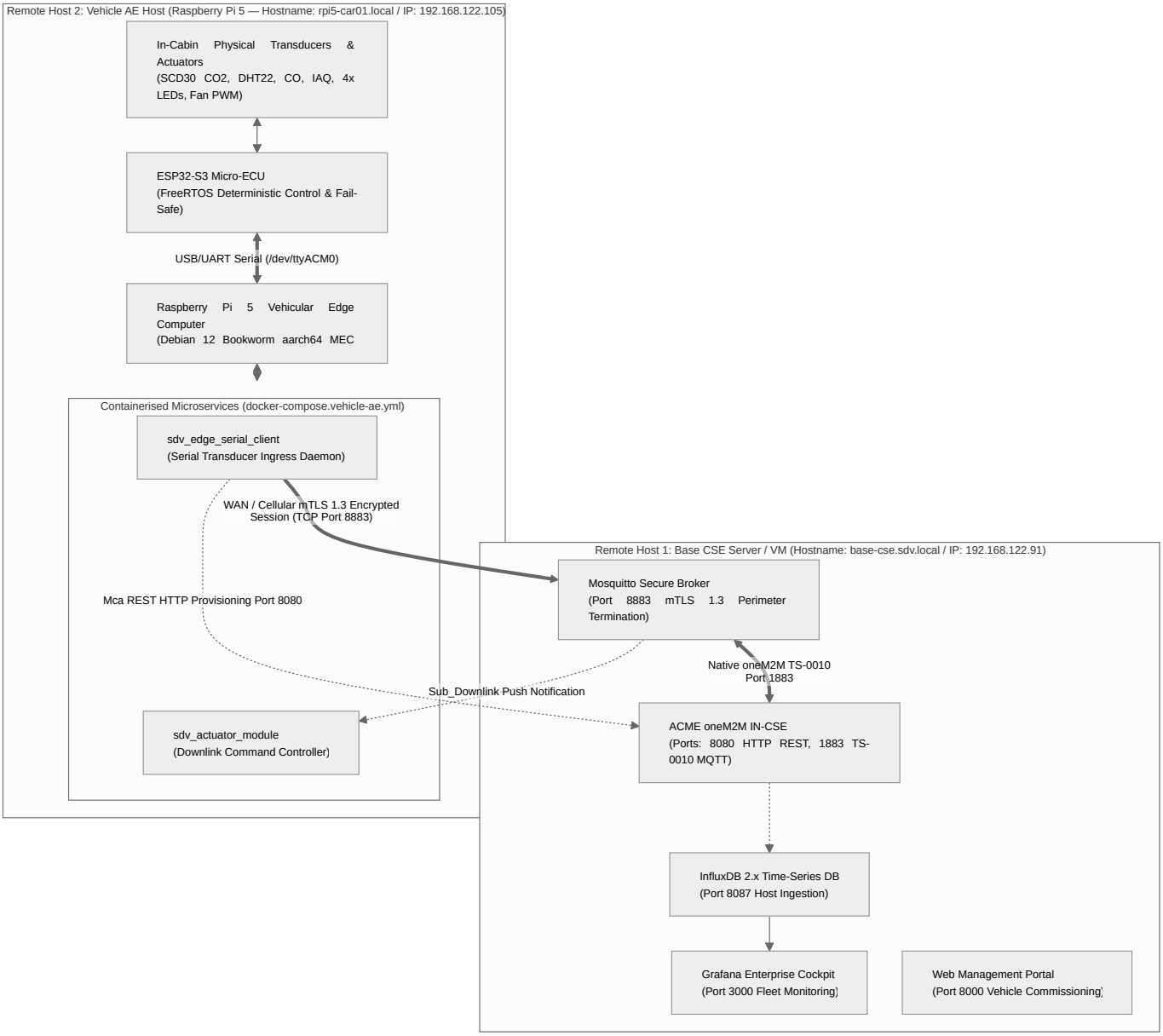
Attribute	Remote Host 1: Base CSE Platform	Remote Host 2: Vehicle AE Platform
Physical Hardware	High-performance enterprise server or virtual machine (x86_64 / aarch64, ≥ 8 cores, $\geq 16\,\mathrm{GB}$ RAM).	In-vehicle Raspberry Pi 5 single-board computer (Broadcom BCM2712 quad-core Cortex-A76 @ 2.4 GHz, 8 GB LPDDR4X).
Host Identification	Configured via `BASE_CSE_HOST` (IP address e.g. `192.168.122.91` or FQDN e.g. `base-cse.sdv.cluster.local`).	Configured via `VEHICLE_AE_HOST` (IP address e.g. `192.168.122.105` or mDNS hostname e.g. `rpi5-car01.sdv.vehicle.local`).
Operating System	Ubuntu Server 22.04 / 24.04 LTS (kernel 6.x).	Raspberry Pi OS (Debian 12 Bookworm 64-bit aarch64).
Service Cardinality	**Strict singleton ($C = 1$)** across the fleet.	**Multi-instance (N physical vehicles)**, one per vehicle.
Microservice Stack	Containerised: `sdv_onem2m_cse` , `sdv_mosquitto_broker` , `sdv_bridge_ae` , `sdv_influxdb` , `sdv_grafana` , `sdv_web_portal` .	Containerised edge stack: `sdv_edge_serial_client` (serial ingress to Base CSE) and `sdv_actuator_module` (downlink command listener & actuator controller).
Physical Interfaces	Gigabit / 10G Ethernet NIC to datacenter or cloud VPC.	GPIO, I2C, SPI, UART/USB connected to in-cabin ESP32-S3 and transducers.
Ingress Firewall	TCP 8883 (mTLS), 8080 (REST), 3000 (Grafana), 8000 (Portal), 8087 (TSDB).	**Zero ingress ports required**. All connections are outbound from vehicle to central server.
Deployment Command	`make deploy-base-cse BASE_HOST=192.168.122.91` (or `make menu` Option `[2]`)	`make deploy-vehicle-ae BASE_HOST=192.168.122.91 VEHICLE_ID=vehicle-001` (or `make menu` Option `[3]`)

8.2.1 Remote Host SSH Management and Login Credential Contracts

When orchestrating a distributed deployment from a centralized engineering workstation or continuous integration pipeline, remote provisioning requires explicit SSH login credentials and access parameters for each target machine:

Parameter Key	Target System	Default Value	Description & Operational Security Scope
`BASE_CSE_HOST`	Remote Host 1	`192.168.122.91`	Hostname or IPv4/IPv6 address of the central server/VM hosting the Base CSE.
`BASE_CSE_USER`	Remote Host 1	`${USER}` (`mosud`)	Login username possessing Docker group or sudo privileges on Remote Host 1.
`BASE_CSE_SSH_PORT`	Remote Host 1	`22`	TCP port for SSH management on the Base CSE server/VM.
`BASE_CSE_SSH_KEY`	Remote Host 1	`~/.ssh/id_ed25519`	Path to the private key used for passwordless public-key authentication.
`BASE_CSE_REMOTE_DIR`	Remote Host 1	`~/HVAC-iot-monitoring`	Working directory where Base CSE deployment manifests and scripts reside.
`VEHICLE_AE_HOST`	Remote Host 2	`192.168.122.145`	Hostname or IPv4/IPv6 address of the in-vehicle Raspberry Pi 5 SBC.
`VEHICLE_AE_USER`	Remote Host 2	`${USER}` (`mosud`)	Login username on the Raspberry Pi 5 single-board computer.
`VEHICLE_AE_SSH_PORT`	Remote Host 2	`22`	TCP port for SSH management on the Raspberry Pi 5.
`VEHICLE_AE_SSH_KEY`	Remote Host 2	`~/.ssh/id_ed25519`	Path to the private key used for passwordless public-key authentication.
`VEHICLE_AE_REMOTE_DIR`	Remote Host 2	`~/HVAC-iot-monitoring`	Working directory where Vehicle AE daemon scripts and client certs reside.

The remote orchestrator (`deploy/deploy_remote.sh`) executes pre-flight reachability probes (`nc -z -w 2`), validates public-key authentication (`ssh -o BatchMode=yes`), synchronises deployment assets (`rsync` with universal POSIX `tar` / `scp` streaming fallback for minimal and embedded SBC operating environments lacking `rsync`), and initiates execution remotely on each respective target host.



8.3 Network Routing, Security Demarcation, and Firewall Rules

The distributed topology implements a strict **unidirectional session initiation model**: the vehicular Raspberry Pi 5 node (Host 2) initiates all network sessions towards the central Base CSE server (Host 1). No inbound ports are opened on the vehicular gateway, eliminating external network attack surfaces on the vehicle CAN/LIN bus or in-cabin compute infrastructure.

Flow Identifier	Source Host	Destination Host	Target Port	Protocol & Security	Functional Purpose
F-01: Ingress Mca	Vehicle Host 2	Base CSE Host 1	TCP 8080	HTTP REST (oneM2M TS-0004)	Initial `<m2m:ae>` registration, container reconciliation, and audit.
F-02: Telemetry Stream	Vehicle Host 2	Base CSE Host 1	TCP 8883	oneM2M TS-0010 over mTLS 1.3	Continuous publishing of telemetry ContentInstances to `/oneM2M/req/...`.
F-03: Downlink Actuation	Base CSE Host 1	Vehicle Host 2	Established TCP 8883	oneM2M TS-0010 over mTLS 1.3	Push notification of closed-loop HVAC regulation commands via MQTT subscription.
F-04: Operations UI	Administrator	Base CSE Host 1	TCP 3000	HTTPS / HTTP Web	Grafana operational telemetry dashboard access.
F-05: Commissioning UI	Fleet Engineer	Base CSE Host 1	TCP 8000	HTTPS / HTTP Web	Web Portal single-vehicle commissioning and dynamic decommissioning.
F-06: Database Storage	Internal Mesh	Base CSE Host 1	TCP 8087 (host) / 8086 (int)	HTTP REST API	Bridge AE persisting normalised sensor frames to InfluxDB.

8.4 Environment and Configuration Contracts

8.4.1 Base CSE Infrastructure Host Environment (`deploy/.env`)

Configured on Remote Host 1 (Server / VM):

```
# ================================================================================
# Base CSE Infrastructure Platform Configuration
# Host 1: Central Server / VM (Singleton Instance)
# ================================================================================

DEPLOY_TARGET=remote
BASE_CSE_HOST=192.168.122.91
BASE_CSE_USER=mosud
BASE_CSE_SSH_PORT=22
BASE_CSE_SSH_KEY=~/.ssh/id_ed25519
BASE_CSE_REMOTE_DIR=~/HVAC-iot-monitoring

VEHICLE_AE_HOST=192.168.122.145
VEHICLE_AE_USER=mosud
VEHICLE_AE_SSH_PORT=22
VEHICLE_AE_SSH_KEY=~/.ssh/id_ed25519
VEHICLE_AE_REMOTE_DIR=~/HVAC-iot-monitoring

# Service Addressing
GATEWAY_IP=192.168.122.91
CSE_HOST=192.168.122.91
MQTT_HOST=192.168.122.91
INFLUXDB_HOST=192.168.122.91

# Service Port Allocations
ONEM2M_PORT=8080
MOSQUITTO_PORT=8883
MOSQUITTO_PLAIN_PORT=1883
INFLUXDB_HOST_PORT=8087
INFLUXDB_INTERNAL_PORT=8086
GRAFANA_PORT=3000
BRIDGE_AE_PORT=5000
WEB_PORTAL_PORT=8000

# oneM2M Core Parameters
ONEM2M_CSE_ID=id-sdv
ONEM2M_CSE_NAME=sdv-cse
ONEM2M_ADMIN_ORIGIN=CAdmin
ONEM2M_RVI=3

# Target oneM2M Endpoints
BASE_URL=http://192.168.122.91:8080
NOTIFICATION_BASE_URL=http://192.168.122.91:8080
DOWNLINK_NOTIFICATION_URL=http://192.168.122.91:8080/downlink
```

8.4.2 In-Vehicle Raspberry Pi 5 Environment (`vehicle/.env` per vehicle)

Configured on Remote Host 2 (In-Vehicle Raspberry Pi 5):

```
# ================================================================================
# In-Vehicle Application Entity (AE) Configuration
# Host 2: In-Vehicle Raspberry Pi 5 (Instance Car01)
# ================================================================================

DEPLOY_TARGET=remote
BASE_CSE_HOST=192.168.122.91
VEHICLE_AE_HOST=192.168.122.145

# Vehicle Hardware & Cryptographic Identity
VEHICLE_VIN=VIN-WAUZZZ-SDV-2026-0042
VEHICLE_NODE_ID=sdv-car-0042
CLIENT_CERT_PATH=/etc/sdv/certs/client_esp32.crt
CLIENT_KEY_PATH=/etc/sdv/certs/client_esp32.key
CA_CERT_PATH=/etc/sdv/certs/ca.crt

# Target Central Base CSE Server / VM Connection
BASE_URL=http://192.168.122.91:8080
CSE_RN=sdv-cse
ORIGINATOR=CAdmin
APP_ID=N.org.fhtw.sdv.cabin
RVI=3

# Local In-Vehicle Instrumentation Configuration
LOCAL_ESP32_PORT=/dev/serial/by-id/usb-Espressif_ESP32-S3_001
DOWNLINK_NOTIFICATION_URL=mqtt://192.168.122.91:8883/oneM2M/req/id-sdv/C_ae_vin_wauzzz_0042/json
```

9. QUALITY ATTRIBUTES AND VERIFICATION MATRIX

Quality Attribute	Requirement Specification	Verification Method
Singleton Guarantee	Exactly one Base CSE exists per network domain on the Server/VM. Zero duplicate CSEBase roots permitted.	Unit & integration test `make validate-base-cse` verifying singular `<m2m:cb>`.
AE Scalability	Fleet capacity scales to N independent vehicle AEs on distributed Raspberry Pi 5 units without code modifications.	Iterative test harness executing onboarding for M arbitrary VINs.
Collision Immunity	Two vehicles with different VINs must never claim or overwrite the same AE tree on the Base CSE.	Ingestion collision test asserting HTTP 409 / 403 on duplicate registration attempts.
Zero Initial State	Fresh deployment of Base CSE on the Server/VM must start with zero vehicular AEs.	Preflight audit confirming `len(cb.childResource[ty==2]) == 0`.
Native Ingestion	Zero intermediary proxy containers; native oneM2M TS-0010 MQTT / Mca HTTP.	Integration test verifying Node-RED absence and direct primitive routing.
Fail-Safe Containment	Network disconnection of Vehicle A's Raspberry Pi 5 has zero impact on Vehicle B's ingestion loop.	Network fault injection testing with `inject_faults.py`.